

Voici quelques-unes des commandes Git les plus courantes et utiles

1. Configuration de Git

- `git config --global user.name "Votre Nom"` → Configure votre nom.
- `git config --global user.email "votre.email@example.com"` → Configure votre email.
- `git config --list` → Affiche la configuration actuelle.

2. Initialisation et Clonage

- `git init` → Initialise un nouveau dépôt Git.
- `git clone <url>` → Clone un dépôt distant sur votre machine.

3. Gestion des fichiers

- `git status` → Vérifie l'état des fichiers.
- `git add <fichier>` → Ajoute un fichier à l'index.
- `git add .` → Ajoute tous les fichiers modifiés.
- `git commit -m "Message du commit"` → Valide les modifications avec un message.
- `git rm <fichier>` → Supprime un fichier et l'ajoute à la liste des modifications.
- `git mv <ancien> <nouveau>` → Renomme un fichier.

4. Historique et Journal

- `git log` → Affiche l'historique des commits.
- `git log --oneline --graph` → Affiche un historique simplifié et graphique.
- `git show <commit>` → Affiche les détails d'un commit spécifique.
- `git diff` → Compare les modifications non indexées.

5. Branches

- `git branch` → Liste les branches.
- `git branch <nom>` → Crée une nouvelle branche.
- `git checkout <nom>` → Bascule vers une branche.
- `git switch <nom>` → Alternative à `checkout` pour changer de branche.
- `git merge <nom>` → Fusionne une branche dans l'actuelle.
- `git branch -d <nom>` → Supprime une branche.

6. Synchronisation avec un dépôt distant

- `git remote -v` → Liste les dépôts distants.
- `git remote add origin <url>` → Associe un dépôt distant.
- `git push origin <branche>` → Pousse les commits vers le dépôt distant.
- `git push -u origin <branche>` → Définit la branche distante par défaut.

- `git pull origin <branche>` → Récupère et fusionne les modifications distantes.
- `git fetch origin` → Récupère les modifications sans fusionner.

7. Réinitialisation et Correction

- `git reset --hard <commit>` → Réinitialise tout l'historique à un commit donné.
- `git reset --soft <commit>` → Réinitialise à un commit sans perdre les modifications locales.
- `git revert <commit>` → Annule un commit en créant un nouveau commit inverse.
- `git stash` → Sauvegarde temporairement les modifications non validées.
- `git stash pop` → Récupère les modifications enregistrées avec `stash`.

Besoin d'une commande spécifique ou d'une explication plus détaillée ? 😊